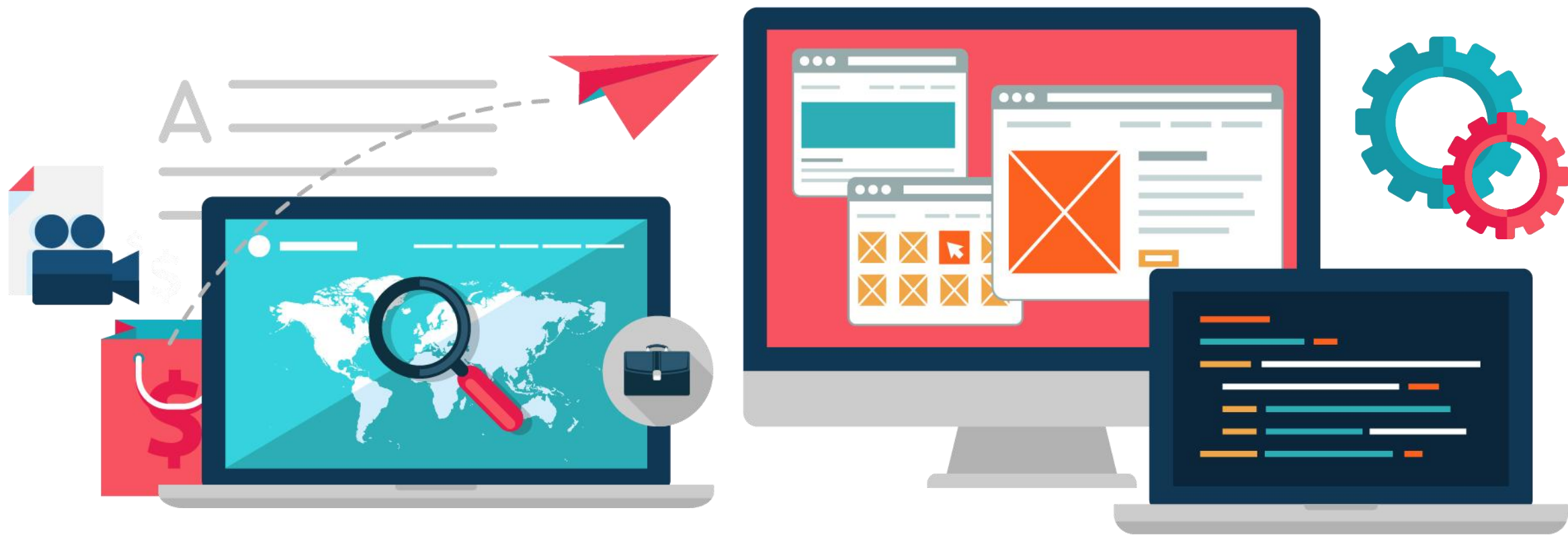


面向对象程序设计

Object Oriented Programming





类间关系



01

编译期依赖性

02

类间逻辑关系

03

关联

04

聚集(组合和聚合)

05

依赖

06

类间关系-例





```
class A {
    //....
};

class B {
public:
    void f(A& a);
private:
    A * pa;
};
```

单向联系



```
class B;
class A {
    B * pB;
};

class B{
public:
    void f( A * p);
    A  g( );
private:
    A * pa;
};
```

双向联系

代码设计时尽可能采用单向联系，以降低耦合性



a.h必须包含b.h才能编译成功，则A与B之间存在强关联。

例1

//a.h

#include "b.h" //前置声明 class B;不行

class A

{

 B mB;

};

结论：尽可能不使用对象成员，而用指针成员，从而可以通过前置声明来降低编译期依赖性。

例2

// a.h

#include "b.h"

class A {

 public: void f (B* pB) {

 return pB->g();

 } // 内联实现增加编译期依赖性

};

结论:直接使用内联方式定义成员函数，可能增强文件的依赖性，可使用前置声明和外联实现避免。



A和B类间有双向联系，且两者之间有强关联，那么A与B之间存在硬关联。依赖性较强，应尽量避免。

```
//a.h
#include "b.h"
class A {
public:
    A( );
    A(int );
private:
    B mB;
};
```

```
// b.h
class A;
class B {
public:
    B(int);
    A Foo( );
private:
    int mData;
};
```



a.cpp中必须包含b.h才能编译通过，则A与B之间存在弱关联。

```
//b.h  
class B {  
public:  
    B( );  
    B(int );  
private:  
};
```

```
// a.h  
class B;  
class A {  
public:  
    A(int);  
    B Func( );  
private:  
    int mData;  
};
```

```
//a.cpp  
#include "a.h"  
#include "b.h"  
B A::Func( )  
{  
    return B(mData);  
}
```

结论: 最常见的关联形式



A类只使用了B类的指针或引用，那么A与B之间存在软关联。编译期依赖性最低，优先考虑。

```
// a.h  
  
class B;  
  
class A {  
    public:  
        A( B * pb);  
        B & func01( );  
        B * func02( B & aB );  
  
    private:  
        B * mpB;  
};
```

结论: 使用指针型数据成员，函数参数以指针或引用形式传递，函数返回时尽量避免直接返回值对象。

即尽可能指针或引用，而不是对象形式

类间的联系依次减弱

1. 继承：类间的垂直关系
2. 硬联系(硬关联)
3. 强联系(强关联)
4. 弱联系(弱关联)
5. 软联系(软关联)

◆ 自定义类型的数据成员：

- ◆ 使用指针形式，RAII、三/五法则、零法则
- ◆ 引用形式，显式初始化、缺省赋值函数delete、绑定不可变更(const指针)，需避免悬垂引用
- ◆ 对象形式，完整类型、编译期依赖性强、不能是接口类和抽象类

◆ 函数的自定义类型参数

- ◆ 使用指针和引用形式，注意区别

◆ 函数返回的自定义类型

- ◆ 尽可能不使用对象形式，特殊：小巧且语义为值的类，如Point

◆ 适当使用前置声明和外联实现



垂直方向：

◆泛化

◆实现

水平方向：

◆关联

◆依赖



1. 数据成员形式: `class B { A * mpA };`

2. 函数参数:

```
class B {  
    void Func( A * pa);  
};
```

3. 函数返回值:

```
class B {  
    A* Func01( );  
};
```

4. 函数实现:

```
int B::Func02(int n ) {  
    A * pa = new A;  
    int val=pa->g( ) +n;  
    delete pa;  
    return val;  
}
```

关联

“knows a”关系，表示类B知道/了解类A，一般是长期性的，比依赖更强

依赖

“uses a”关系，表示类B使用到了类A，这种使用关系是具有偶然性的临时性的非常弱的，但是类A的变化会影响到类B



代码形式:

```
class B {  
public: ....  
public/protected/private:  
    A * mpA;  
};
```

◆可单向、可双向

◆含义：在B类对象的生存期，“一直知道” A类对象。

◆一般关联：强调非偶然性的“知道”。

- 一般关联
- 自关联
- 关联类

◆聚集关联：强调整体-部分关系

- 聚合：
- 组合：

```
class University;  
class Course;
```

```
class Student {  
public:
```

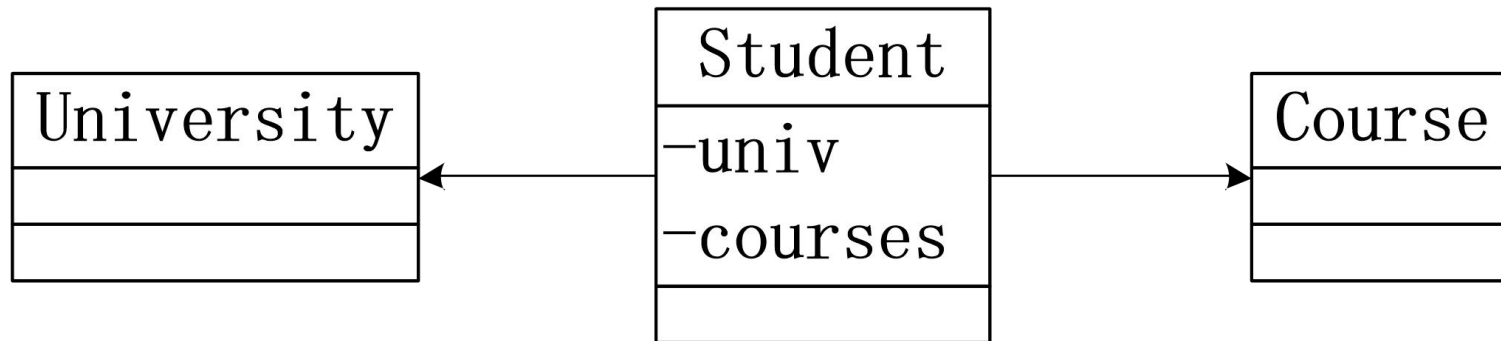
```
....
```

```
private:
```

```
    University * univ;
```

```
    Course * courses[30];
```

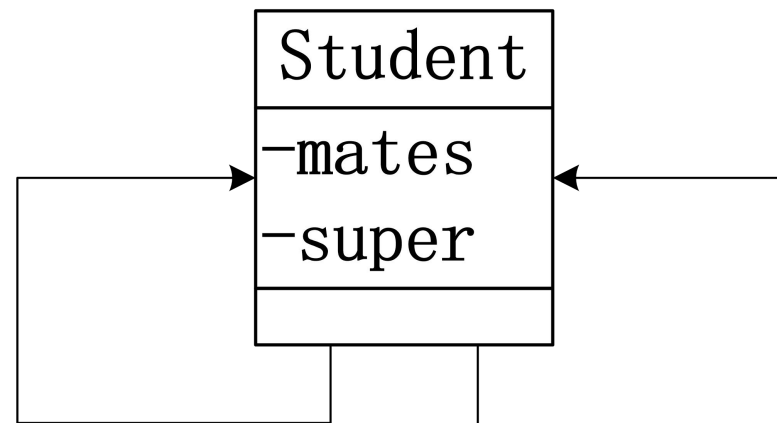
```
};
```



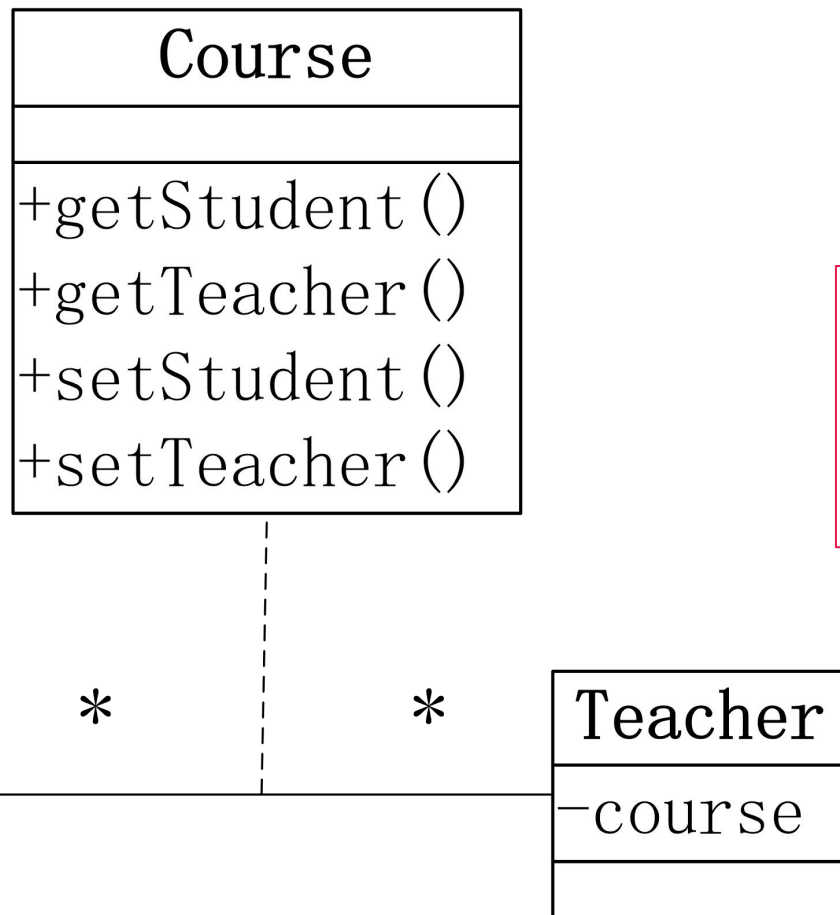
◆一般关联：强调非偶然性的“知道”。

- 可单向，可双向
- 可一对一，
- 可一对多，多对一
- 可多对多

```
class Student {  
public:  
    ....  
private:  
    Student    * mates[30];  
    Student    * super;  
};
```



◆ 自关联：关联本类对象



◆关联类：将关联关系抽象成一个/多个独立的关联类，使得关联类既表示关联关系，也表示关联对象。



代码形式:

```
class B {  
public: ....  
public/protected/private:  
    A * mpA;  
};
```

一般关联: 强调非偶然性的“知道”。

- 一般关联
- 自关联
- 关联类

◆聚集关联: 强调整体-部分关系

◆ 一般关联关系中两个类是处于相同的层次，而聚集关联关系中两个类是处于不平等的层次，一个表示整体，一个表示部分。

◆ 例如:

- | | |
|---------------------|-------------------|
| ◆ 自行车和车把、响铃、轮胎; | ◆ 公司和部门; |
| ◆ 汽车和引擎、轮胎、刹车装置; | ◆ 人和大脑、四肢; |
| ◆ 计算机和主板、CPU、内存、硬盘; | ◆ 窗口和标题栏、菜单栏、状态栏。 |

◆聚合/组合关系：特殊的关联，通常表示整体与部分的关系

- 聚合关系： B “has a” A, 但B类不负责A类对象的生存与消亡。

- 组合关系： B “contains a” A, B类负责A类对象的生存与消亡。

◆聚合、组合是逻辑关系，要区别于现实世界中的事物关系

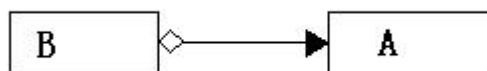
- 现实世界中：

 - ◆计算机有网卡叫计算机，没有网卡也可叫计算机，聚合关系更合适。

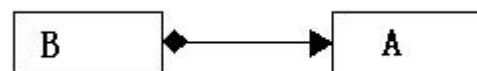
 - ◆人必须有大脑；部门是由公司来建立和取消的。组合关系更合适。

- 程序设计中：

 - ◆针对问题域，选择适当关系模型



聚合的UML表示



组合的UML表示



一般关联、聚合和组合关系，代码形式上的比较

//一般关联关系：强调非偶然性的“知道”

```
class A;
class B {
public:
    B(A * pa):mpA(pa) {}
private:
    A * mpA;
};
```

//聚合关系：强调整体-部分关系，但不负责成员的创建和销毁

```
class A;
class B {
public:
    B(A * pa):mpA(pa) {}
private:
    A * mpA;
};
```

说明：聚合关系中，A是否生存与B无关。

//组合关系:强调整体-部分关系，且负责成员的创建和销毁

```
class A {
    public: A(int n);};
class B {
public:
    B(int n ) {
        mpA = new A(n); }
    ~B( ) {
        delete mpA;}
private:
    A * mpA;
};
```

说明:组合关系中，B控制A的生死(同生共死)

//组合关系:强调整体-部分关系，且负责成员的创建和销毁

```
class A {
    public: A(int n);};
class C {
public:
    CreateA(int n ) {
        mpA = new A(n); }
    DeleteA( ) {
        delete mpA;}
private:
    A * mpA;
};
```

说明:组合关系中，B控制A的生死(非同生共死)

1. 数据成员形式: `class B { A * mpA };`

2. 函数参数:

```
class B {  
    void Func( A * pa);  
};
```

3. 函数返回值:

```
class B {  
    A* Func01( );  
};
```

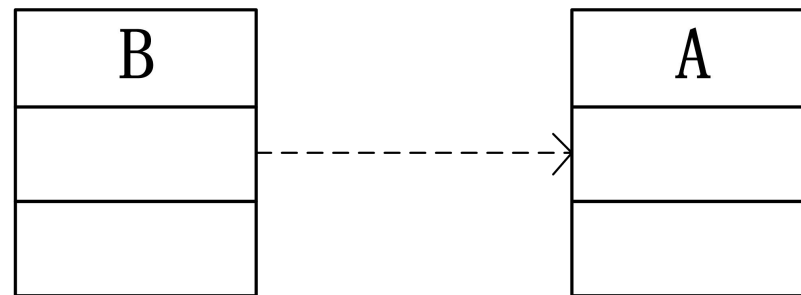
4. 函数实现:

```
int B::Func02(int n ) {  
    A * pa = new A;  
    int val=pa->g( ) +n;  
    delete pa;  
    return val;  
}
```

◆ 可以简单的理解，就是一个类B使用到了另一个类A，并且这种使用关系是具有**偶然性的、临时性的、非常弱的**，A类的变化会影响到B。

◆ 例如：机器生产零件，充电电池通过充电器来充电，自行车通过打气筒来充气，人借助螺丝刀拧螺丝，人借用船过河，艺术家鉴赏艺术品，警察抓小偷，小猫钓鱼，学生读书，某人买车。

◆ 表现在代码层面，为类A作为参数被类B在某个方法中使用，例如：局部变量，方法中的参数、返回值，对静态方法的调用等。



例：

老鼠在苹果园中吃苹果，老鼠每吃一个苹果，就根据该苹果的所含能量增加一些体重。设计老鼠和苹果类。

分析：

- 老鼠不需要“一直知道”苹果，苹果也不需要“一直知道”老鼠，不适合关联关系，依赖关系即可。
- 吃的行为由两个对象共同参与（苹果和老鼠），
- 放苹果类中，为被吃；
- 放老鼠类中，为吃；

核心功能（Eat函数）：**尽可能将其放在主动的多变的类中**

例如大老鼠、小老鼠，或者猴子、熊猫、大象等吃苹果

例如老鼠不仅吃苹果，还吃西瓜、雪梨、草莓等

行为的主体易变，则放在主体侧，行为的受体易变，则放在受体侧

//苹果类:

```
class Apple {  
public:  
    Apple(int e):power(e) { }  
    int getEnergy( ) const  
        { return power; }  
private:  
    int power;  
};
```

//老鼠类

```
class Mouse {  
public:  
    Mouse(int w):weight(w) { }  
  
    int  getWeight( ) const {return weight;}  
  
    void eat(Apple & one)  
        { weight += one.getEnergy() * 0.5; }  
private:  
    int weight;  
};
```

eat和eatenBy也可同时存在:

//苹果类:

```
class Apple {  
public:  
    Apple(int e):power(e) { }  
    int getEnergy( ) const  
        { return power; }  
  
    void eatenBy(Mouse & m) {  
        int w = m.getWeight( );  
        m.setWeight(w+power*0.5);  
    }  
private:  
    int power;  
};
```

//老鼠类

```
class Mouse {  
public:  
    Mouse(int w):weight(w) { }  
  
    int  getWeight( ) const  
        { return weight; }  
    void setWeight(int w) { weight =w; }  
  
    void eat(Apple & one)  
        { one.eatenBy(*this); }  
private:  
    int weight;  
};
```



类之间逻辑关系所表现的强弱程度依次为：

泛化/实现 > 组合 > 聚合 > 一般关联 > 依赖；



例：每个学生的信息除了包括姓名、学号等之外，还要有宿舍信息。宿舍信息包括几号楼，第几层，几号房间，以及住了哪几个学生等信息。

宿舍和学生间的关系：

- ◆ 现实世界中：依赖普通关联或聚合关系(认知常识)
- ◆ 程序设计中：还要考虑问题域的关注点
 - ◆ 普通“知道”，如通信录
 - ◆ 以学生为中心，如学生管理程序。
 - ◆ 以宿舍为中心，如宿舍管理程序。



强调学生知道“地址”信息，学生可以走读、住校、

```
class Student {
public:
    Student(Dorm * aDorm): mpDorm(aDorm) { }
    const char * Address( ) const {
        if (mpDorm)
            return mpDorm->Floor( );
        else
            return "";
    }
private:
    Dorm * mpDorm;
    ....
};
```



宿舍是学生信息的一个必要组成部分，虽然学生**可以没有宿舍**
这里强调部分--整体关系

```
class Student {  
public:  
    Student(Dorm * aDorm): mpDorm(aDorm) { }  
    const char * Address( ) const {  
        if (mpDorm)  
            return mpDorm->Floor( );  
        else  
            return "";  
    }  
private:  
    Dorm *    mpDorm;  
    ....  
};
```



宿舍是学生信息的一个必要组成部分，**学生必须有宿舍**
这里强调部分--整体关系

```
class Student {  
public:  
    Student(): mpDorm(new Dorm) { }  
    ~Student() { delete mpDorm;}  
private:  
    Dorm *    mpDorm;  
    ....  
};
```



- ◆ 普通关联--只强调宿舍“知道”其中的学生
- ◆ 组合关系：宿舍-学生是整体-部分的关系，且宿舍负责创建学生，销毁学生
- ◆ 聚合关系：宿舍-学生是整体-部分的关系，且宿舍不负责创建学生，也不负责销毁学生



```
class Dorm {  
    public:  
        Dorm( ) {  
            for(int i=0;i<4;i++)  
                mStudents[i] = new Student;  
        }  
        ~Dorm( ) {  
            for(int i=0;i<4;i++)  
                delete mStudents[i];  
        }  
    private:  
        Student * mStudents[4];  
};
```



```
class Dorm {
public:    Dorm( Student * s[ ],int count):maxCount(count) {
            mStudents = new Student*[maxCount];
            for(int i=0;i<count;i++)
                mStudents[i] = s[i];
        }
        ~Dorm( )    { delete[ ] mStudents;    }
        void AddStudent(Student * s, int index){
            if ( mStudents[index] == nullptr))
                mStudents[index] = s;
        }
        void RemoveStudent(int index)
            { mStudents[index] = nullptr;    }

private:
        int            maxCount;
        Student **    mStudents;

};
```



本章结束